

V Y O M I R A T E C H

Scaling beyond the binary

Cloud & DevOps Engineering Fellowship

*Not a Coaching Centre. A Live Technical Firm.
Learn from engineers who are actively building in production.*

15 SESSIONS | 3 HOURS EACH | 100% LIVE | DIRECT REFERRALS

Not a Coaching Centre. A Live Technical Firm.

Everything in this program is on YouTube for free. We know that.

What YouTube cannot give you: an active industry engineer stopping you mid-lab when your approach is wrong. Real-time correction. Live interview practice with actual feedback. And at the end of it - a personal referral, not a certificate.

Every session is live. Every lab is instructor-reviewed. Every cohort is small enough that your instructor knows your name.

Program at a Glance

DURATION 15 Sessions <i>3 hours each 45 hours total</i>	FORMAT 100% Live <i>No recordings - real instructors, no pre-recorded lectures, real feedback</i>	PREREQUISITE Basic Understanding <i>Any language No cloud experience needed</i>
PROJECTS BUILT 4 Projects <i>Each one goes on your GitHub portfolio</i>	INTERVIEW PREP 60+ Questions <i>Practised live every session with feedback</i>	CAREER TARGET DevOps Engineer <i>Junior Cloud Engineer Systems Engineer SRE</i>

How Every 3-Hour Session Is Structured

The last 20 minutes of every session are live interview practice. Not optional. Not homework. Part of the session.

30 MIN Concept Briefing Real company scenario, not a textbook definition. Instructor explains where and why this is used in production.	90 MIN Engineering Lab You build it yourself. Instructor watches, corrects mistakes in real time before they become habits.	40 MIN Lab Debrief The correct approach, the most common mistakes, and what a senior engineer would do differently.	20 MIN Interview Round Answer 3-4 questions out loud. Instructor grades every answer and tells you exactly what to say instead.
---	---	---	---

Technology Stack

Industry-standard tools used by real engineering teams in 2025 production environments.

>> Linux / Bash <i>Server foundation</i>	>> Python <i>Cloud automation</i>	>> Git + GitHub <i>Version control</i>	>> Docker <i>Containerisation</i>
>> Kubernetes <i>Orchestration</i>	>> AWS (EC2/S3/IAM/VPC) <i>Cloud infrastructure</i>	>> Terraform <i>Infrastructure as Code</i>	>> GitHub Actions <i>CI/CD pipelines</i>
>> Prometheus <i>Metrics collection</i>	>> Grafana <i>Observability dashboards</i>	>> Nginx <i>Reverse proxy / web</i>	>> PostgreSQL <i>Production database</i>

A Real DevOps Workflow, Made Tangible

1 Code Push Developer commits	2 CI Pipeline Auto-triggered test + build	3 Docker Build Image built + scanned	4 Cloud Deploy Terraform + Kubernetes	5 Monitor Prometheus + Grafana live
---	---	--	---	---

Your 15-Session Learning Journey

Each phase builds directly on the previous one. By Session 15, you have not learned 15 separate topics - you have built one complete, working system.

Foundation	Containers & Cloud	CI/CD & IaC	Production Ready
01 > 02 > 03 > 04	05 > 06 > 07 > 08	09 > 10 > 11	12 > 13 > 14 > 15

Detailed Curriculum

PHASE 1 Sessions 1-4

The Foundation

Linux, Git, Bash, Python. The skills every interview tests first.

Session 01 Linux & The Command Line How a Server Really Works

The questions every DevOps interviewer asks in round one.

- > Why Linux: 96% of cloud VMs, all containers, every DevOps tool
- > Filesystem hierarchy: /, /etc, /var/log, /home, /bin
- > Essential daily commands: ls, cat, grep, find, ps, top, chmod
- > File permissions: reading rwxr-xr-x, chmod 755 vs 644
- > Processes: ps aux, systemctl, kill vs kill -9
- > Package management: apt update, apt install, service verification

Lab: Your First Server

- SSH into an Ubuntu VM, navigate filesystem, set permissions
- Find top 3 CPU processes using ps and top
- Install Nginx, verify running, confirm page live in browser

Tools: Linux SSH Nginx Bash systemctl chmod

Session 02 Git & Version Control - The Way Teams Write Code Together

Not just git commit - the branching, merging, and conflict resolution used in every real team.

- > The Git data model: working dir -> staging -> repository
- > Core workflow: init, add, commit, status, log, diff
- > Branching: checkout -b, merge, conflict resolution
- > GitHub: push, pull, clone, pull requests
- > Undoing mistakes: restore, revert vs reset
- > .gitignore: preventing secrets from ever reaching GitHub

Lab: Collaborative Git Workflow

- Create repo, feature branch, deliberate merge conflict - resolve it
- Push to GitHub, open a Pull Request, merge it

Tools: Git GitHub Branches Pull Requests .gitignore file

Session 03 Bash Scripting - Automating the Work You Do Every Day

The difference between a DevOps engineer and someone who types the same commands repeatedly.

- > Variables, arguments, default values (\${1:-80})
- > Conditionals: if/elif/else, file checks (-f, -d, -z)
- > Loops: for over arrays and files, while with retry logic
- > Functions: local variables, return codes (0=success)
- > Error handling: set -euo pipefail
- > Cron jobs: schedule format, absolute paths, output redirection

Lab: Server Health Check Script

- Write script checking disk %, memory, and Nginx status
- Alert logic with threshold argument, output to log file
- Schedule with cron every 5 minutes

Tools: Bash Cron Shell Scripting set -e Variables

Session 04 Python for DevOps - Scripting That Talks to Cloud APIs

Bash handles files and processes. Python handles APIs, JSON, HTTP, and automation that actually scales.

- > Python essentials for automation: f-strings, lists, dicts, os.environ
- > Reading/writing files: JSON, CSV, YAML with proper file handling
- > HTTP requests: requests library, raise_for_status(), timeout handling
- > boto3: AWS SDK - EC2 inventory, S3 operations, IAM queries

- > Idempotency: check-before-create pattern for safe automation
- > Error handling: specific exceptions, NoCredentialsError, HTTPError

Lab: AWS Resource Inventory Script

- List running EC2 instances with boto3, flag untagged ones
- Export to CSV, accept region as CLI argument

Tools: Python boto3 AWS SDK requests JSON CSV

PHASE 2 Sessions 5-8**Containers & Cloud***Docker and AWS. The two things on every DevOps job description.***Session 05 Docker & Containerisation - What It Actually Is***Containers are not virtual machines. Most freshers say they are. You will know why they are different.*

- > Why containers: the 'it works on my machine' production problem
- > Containers vs VMs: namespaces (isolation) and cgroups (resource limits)
- > Dockerfile: FROM, WORKDIR, COPY, RUN, EXPOSE, CMD - layer order matters
- > Layer cache optimization: requirements.txt before app code
- > Running containers: -d, -p port mapping, --name, volumes, env vars
- > CMD vs ENTRYPOINT: what each does, when to use each

Lab: Containerise a Real Application

- Write Dockerfile for a Python/Node.js app, correct layer order
- Debug a broken Dockerfile with 3 deliberate errors
- Reduce image size with slim base - measure before/after

Tools: Docker Dockerfile Docker Hub Namespaces cgroups**Session 06 Docker Compose- Running Multi-Container Applications***Real applications are never just one container. This session covers how the pieces connect*

- > Multi-container architecture: web, API, database as separate services
- > docker-compose.yml: services, networks, volumes, environment
- > Service communication: container-to-container via service names
- > Data persistence: named volumes vs bind mounts
- > Health checks: depends_on with condition: service_healthy
- > Environment files (.env): keeping secrets out of compose files

Lab: Three-Tier Application Stack

- Run web + API + PostgreSQL via Compose with shared network
- Fix a startup race condition using health check wait logic

Tools: Docker Compose PostgreSQL Networks Volumes .env files

Session 07 AWS Core Services- EC2, S3, IAM, and VPC

The four AWS services that appear in almost every DevOps fresher interview.

- > EC2: instance types, AMIs, key pairs, security groups, SSH access
- > S3: buckets, permissions, static website hosting, IAM policies
- > IAM: users vs roles, least privilege, never use root account
- > VPC: public vs private subnets, internet gateway, route tables
- > Shared responsibility model: what AWS secures vs what you secure
- > AWS CLI: configure, ec2 describe-instances, s3 cp commands

Lab: Host a Real Application on AWS

- EC2 instance with Nginx, S3 static site, IAM role with no access keys
- Security Group blocking all except HTTP - tested and verified

Tools: AWS EC2 AWS S3 AWS IAM AWS VPC Security Groups

Session 08 AWS Networking & Architecture

Connecting the services together the way real companies connect them.

- > VPC deep dive: CIDR blocks, subnet sizing, public vs private isolation
- > NAT Gateway: private subnet internet access without inbound exposure
- > Application Load Balancer: target groups, health checks, routing
- > Auto Scaling Groups: launch templates, min/max/desired, scale policies
- > RDS: managed databases, Multi-AZ, subnet groups, parameter groups
- > Architecture diagrams: drawing and presenting AWS designs

Lab: Resilient Two-Tier Architecture

- ALB + two EC2 in private subnets + RDS - draw diagram first
- Kill one EC2, verify traffic continues with zero downtime

Tools: ALB Auto Scaling RDS NAT Gateway VPC Architecture

PHASE 3 Sessions 9-11**CI/CD & Infrastructure Automation***Code to production automatically. No manual steps.***Session 09** **CI/CD Pipelines - Automated Delivery from Code to Cloud***The pipeline that every company wants, and every DevOps engineer is expected to build.*

- > CI vs CD: catching problems early vs deploying automatically
- > GitHub Actions: workflow YAML, triggers, jobs, steps, runners
- > Pipeline stages: checkout -> install -> test -> build -> push -> deploy
- > Secrets management: GitHub Secrets, never hardcode credentials
- > Docker integration: build image, push to registry via pipeline
- > Debugging pipelines: reading logs, identifying failure points

Lab: End-to-End Deployment Pipeline

- GitHub Actions: push triggers build, Docker push, SSH deploy to EC2
- Push a bug, watch pipeline fail, fix, confirm deployment

Tools: GitHub Actions CI/CD Docker Hub Workflows Secrets**Session 10** **Terraform - Infrastructure as Code***Clicking around the AWS console creates infrastructure no one can reproduce. Terraform fixes that.*

- > IaC principle: infrastructure in Git, reproducible, version-controlled
- > Terraform internals: providers, state file, plan vs apply cycle
- > Resources, variables, outputs: the three building blocks
- > Remote state: S3 backend + DynamoDB locking for team use
- > terraform plan: reading output, understanding destroy warnings
- > State drift: what it is, how to detect it, how to fix it

Lab: Provision AWS with Terraform

- Write Terraform: VPC + subnet + security group + EC2 from code
- Apply, verify in console, destroy - then re-apply identically

Tools: Terraform IaC State Files AWS Provider S3 Backend

Session
11**Kubernetes Fundamentals - Container Orchestration at Scale***When you have many containers to run across many servers, Kubernetes is what every company uses.*

- > Why Kubernetes: running hundreds of containers across many servers
- > Architecture: control plane vs worker nodes, etcd, kubelet, kube-proxy
- > Core objects: Pod, Deployment, Service, Namespace, ConfigMap
- > Self-healing: ReplicaSet ensures desired count is always maintained
- > Services: ClusterIP, NodePort, LoadBalancer - when to use each
- > kubectl daily commands: get, describe, logs, exec, apply, delete

Lab: Deploy and Manage on Kubernetes

- Deploy 3-replica app, kill a pod - watch Kubernetes replace it
- Expose via Service, rolling update to new image version

Tools: Kubernetes kubectl Pods Deployments Services minikube

PHASE 4 Sessions 12-15**Monitoring, Security & Production Readiness***The skills that make you interview-ready and production-trusted.***Session 12** **Monitoring & Observability - Knowing Before Your Users Do***If your application breaks and you find out after your users do, you are a firefighter, not a DevOps engineer.*

- > Why monitoring: knowing before your users do
- > Prometheus: scraping metrics, data model, PromQL basics
- > Grafana: dashboards, panels, data sources, alert rules
- > The RED method: Rate, Errors, Duration - the three signals that matter
- > Alert design: symptom-based alerting vs cause-based alerting
- > Log management: structured logs, centralizing with Loki or CloudWatch

Lab: Production Monitoring Stack

- Deploy Prometheus + Grafana via Docker Compose
- Build RED dashboard, write alert rule - trigger and verify it fires

Tools: Prometheus Grafana Metrics Alerting Loki PromQL**Session 13** **DevSecOps Basics - Security That Does Not Slow You Down***Security is no longer the team that says no. It is the practices you build into every pipeline*

- > DevSecOps: security checks inside the pipeline, not after it
- > Container scanning: Trivy - CRITICAL/HIGH vulnerability gates
- > Secrets detection: git-secrets, preventing credential commits
- > IAM hardening: least privilege, removing wildcard permissions
- > OWASP Top 10: the vulnerabilities DevOps engineers must know
- > Compliance basics: what SOC 2 and ISO 27001 mean day-to-day

Lab: Secure Your Pipeline

- Add Trivy scanning to GitHub Actions - fail on HIGH findings
- Scan repo history for leaked keys, tighten IAM wildcard policies

Tools: Trivy DevSecOps IAM Policies OWASP git-secrets

Session 14 Full Pipeline Integration - Everything Connected End to End

One session where all four phases connect into a single working system.

- > System architecture: drawing the end-to-end diagram and explaining it
- > Integration: Git -> CI/CD -> registry -> Kubernetes -> monitoring
- > Incident response: logs, metrics, recent deploys - methodical approach
- > Rollback: when to roll forward vs roll back, and how to do it fast
- > Runbooks: the written records that let someone else fix it at 2am
- > On-call culture: what being on-call means, how to handle incidents

Lab: End-to-End Integration + Incident Simulation

- Connect all components: push triggers full pipeline to Kubernetes
- Instructor injects bug - detect via Grafana, fix, write postmortem

Tools: Full Stack Incident Response Runbooks Rollback Monitoring

Session 15 Capstone & Mock Interview Day - You Are Now a DevOps Engineer

The final session is not a lecture. It is your job interview.

- > 60 min - Solo Capstone Build: full system from brief, no assistance
- > 60 min - Technical Interview Panel: 2 instructors, every topic covered
- > 60 min - Portfolio Review + Placement Briefing: resume, referrals

Lab: Final Capstone: Full Production System (Solo, 60 min)

- Provision AWS infrastructure with Terraform from scratch
- Containerise app, CI/CD pipeline, Kubernetes deployment, Grafana live

Tools: Terraform Docker GitHub Actions Kubernetes Prometheus AWS

THE CAPSTONE PROJECT

What You Build on Session 15

A live, hiring-ready demonstration of everything in the program - built solo, under time pressure, from a brief.

What the Capstone Is

The capstone is not an assignment you prepare for in advance. It is a 60-minute solo engineering challenge given to you at the start of Session 15.

You receive a production brief describing a fictional company that needs a containerised, automatically deployed, monitored application. You build the entire system from scratch - no help from the instructor, no prior preparation, no scaffolding.

When you finish, you present it. The instructor asks the same questions a technical hiring panel would ask. Your capstone becomes the centerpiece of your portfolio.

What You Build

1	2	3	4	5
Terraform	Docker	GitHub Actions	Kubernetes	Prometheus + Grafana
VPC + EC2 + S3 + IAM provisioned as code	Multi-stage image, scanned, pushed to ECR	CI/CD: build + test + push + deploy	Deployment + Service + rolling update	Metrics scraped, dashboard live

The Evaluation Criteria

Does It Work?	Can You Explain it?	Is It Production-Quality?
- Infrastructure is provisioned correctly - Application is accessible via load balancer - CI/CD pipeline completes without manual steps - Kubernetes shows all pods Running	- Walks through architecture diagram clearly - Explains every design decision unprompted - Handles challenge questions without hesitation - Describes how to debug it if it breaks	- No secrets hardcoded anywhere - IAM follows least privilege - Monitoring shows real application metrics - Rollback procedure is documented

Why the Capstone Matters for Hiring

Most fresher candidates walk into interviews with a list of tools on their resume. When asked to explain a system they have built, they describe something they watched someone else build.

The capstone gives you something different: a real GitHub repository with real infrastructure code, a working CI/CD pipeline with a run history, and a Grafana dashboard with live metrics from an application you deployed yourself.

When a hiring manager asks 'show me something you have built' - you open your laptop and show them.

I N T E R V I E W P R E P A R A T I O N

Built Into Every Session

60+ questions practised live across 15 sessions. Not a mock test at the end - a habit formed throughout.

How Interview Preparation Works

20 min

Every Session

The final 20 minutes of each session are live interview practice on that session's topic. Mandatory. Graded.

60+

Questions Practised

By Session 15 you have answered 60+ real interview questions out loud, with instructor feedback on everyone.

Session 15

Full Mock Panel

A 60-minute technical interview with two instructors covering the entire program. Scored and debriefed.

Written

Feedback Report

Every candidate receives a written evaluation of their mock interview with specific improvement notes.

Types of Questions Practised

Concept Questions

- > What is the difference between a container and a VM?
- > Explain chmod 755 - what can each user do?
- > What does Terraform's state file contain and why is it important?
- > What is the difference between CI and CD?
- > How does Kubernetes restart a crashed pod automatically?

Scenario Questions

- > Production is down. The error rate is spiking. Walk me through what you do first.
- > A developer pushed a secret API key to GitHub. What is your response?
- > Your CI/CD pipeline is taking 45 minutes. How would you investigate and fix it?
- > Walk me through a system you built in this program.
- > How would you roll back a deployment that just broke production?

What Feedback Looks Like

Most interview prep programs mark answers right or wrong. Ours does not.

When you answer a question in the interview round, the instructor tells you three specific things: what was technically correct, what was missing or imprecise, and exactly what a senior engineer would say instead. You hear a model answer after every question.

By Session 15, you have been corrected so many times that the correct answer is instinct, not memory.

AFTER THE PROGRAM

Placement Assistance

Most courses end with a certificate. This one ends with a referral.

What Placement Assistance Means at Vyomira

Portfolio Building

- > 4 GitHub projects built during the program
- > Each has a README, architecture diagram, and documentation
- > Capstone project is the centerpiece - a complete working system
- > CI/CD run history visible - proves the pipeline was actually built
- > Instructor reviews and gives written feedback on every project

Resume Optimisation

- > Resume rewritten around real system builds, not generic skill lists
- > Every bullet quantified: 'Deployed 3-replica Kubernetes cluster' not 'used Kubernetes'
- > LinkedIn profile optimised for DevOps recruiter searches
- > GitHub profile structured to show progression across all 4 projects
- > Cover letter template tailored for DevOps / cloud engineering roles

The Referral

- > Direct introduction to hiring partners by name - not a job board
- > Instructor recommendation based on lab performance and capstone
- > Referral includes a note on specific systems built and strengths

Target Roles & Market

- > DevOps Engineer (fresher to 1 year)
- > Junior Cloud Engineer
- > Site Reliability Engineer Intern
- > Systems Automation Engineer
- > Cloud Support Engineer (AWS/GCP)

- > We refer graduates we would personally vouch for, not everyone
- > Ongoing support until placed - follow-up calls and guidance

- > Startups, SaaS companies, cloud-native product teams

What You Can Demonstrate on Day One of a Job

- > SSH into any Linux server and diagnose issues
- > Write Bash and Python scripts that automate repetitive tasks
- > Containerise an application with a production-ready Dockerfile
- > Deploy and manage AWS: EC2, S3, VPC, IAM, RDS
- > Provision infrastructure with Terraform repeatably
- > Build a CI/CD pipeline that deploys on every push
- > Deploy to Kubernetes and perform rolling updates
- > Set up Prometheus + Grafana monitoring with alerts
- > Explain architecture decisions under interview pressure
- > Show a portfolio of 4 real projects, not course certificates

Apply at vyomiraedu.com

Cohort 2026 | Limited Seats | info@vyomiratech.com